

Building Packages and Tools Using the Compiler API

Jason Bock
Staff Software Engineer
Rocket Mortgage

PRAIRIE DEV CON

WEB | DEV | CLOUD | AI



Personal Info

- <https://jasonbock.net>
- <https://bsky.app/profile/jasonbock.net>
- <https://github.com/jasonbock>
- <https://youtube.com/JasonBock>
- <https://www.linkedin.com/in/jasonrbock/>
- jason.r.bock@outlook.com

Downloads

<https://github.com/JasonBock/Transpire>

<https://github.com/JasonBock/AutoDeconstruct>

<https://github.com/JasonBock/UsingUsings>

<https://github.com/JasonBock/Presentations>

Overview

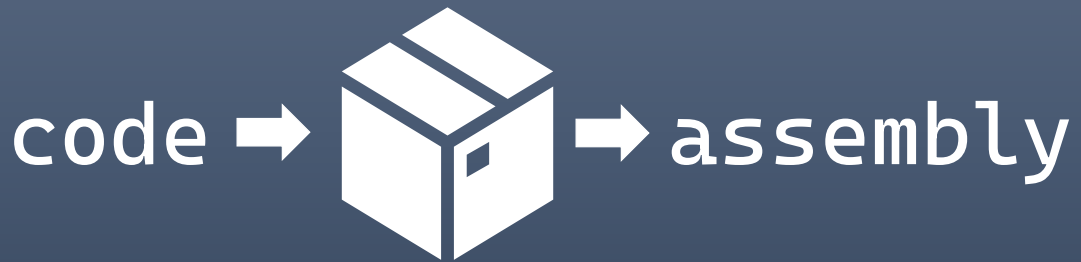
- Compiler API
- Structures
- Demos
- Summary

Remember...

<https://github.com/JasonBock/Transpire>
<https://github.com/JasonBock/AutoDeconstruct>
<https://github.com/JasonBock/UsingUsings>
<https://github.com/JasonBock/Presentations>

Compiler API

Building Packages and Tools Using the Compiler API



Traditionally, compilers have been considered closed boxes. That is, you provide your code to the compiler, it does a whole bunch of stuff, and the result is a product that can be executed on a machine. This was true of C# since 1.0 ... until 2015.



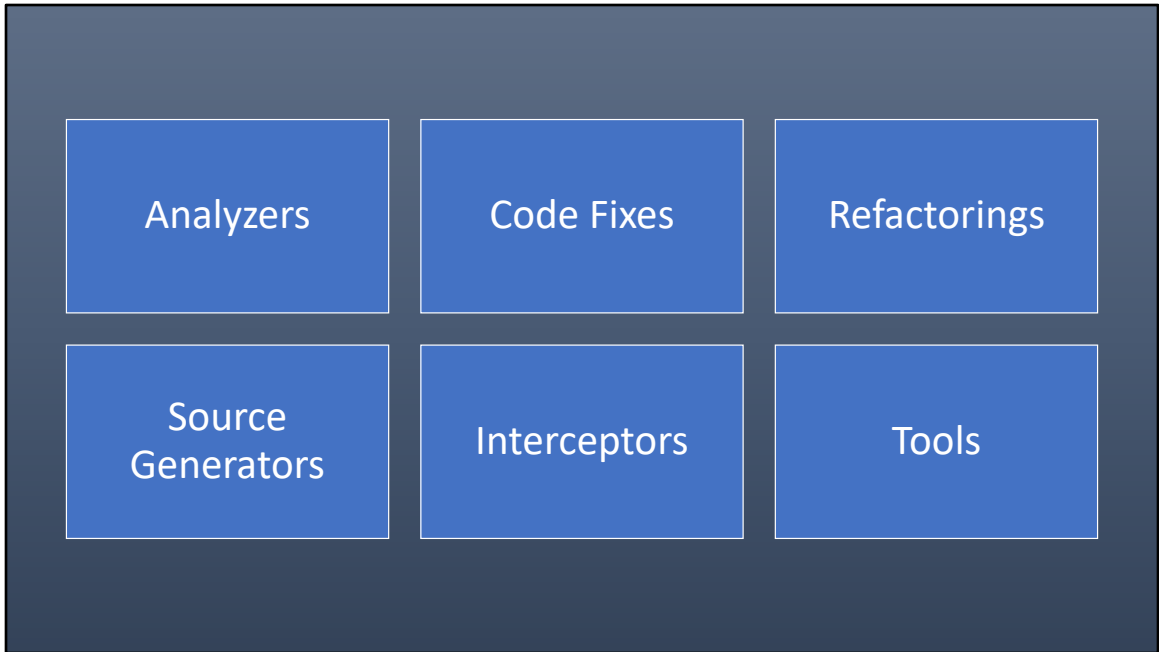
Project Roslyn, or what is officially called the “Compiler API” (packages typically start with “Microsoft.CodeAnalysis”) was a large internal project that started around the release of C# to have C# compiled by C#. This took quite a long to finish, but it was finally released with C# 6. This was also open-sourced, which effectively pushed C# (and .NET in general) to become open-source as well. .NET becoming a cross-platform environment was also becoming a thing at roughly the same time.

<https://user-images.githubusercontent.com/46729679/109719841-17b7dd00-7b5e-11eb-8f5e-87eb2d4d1be9.png>

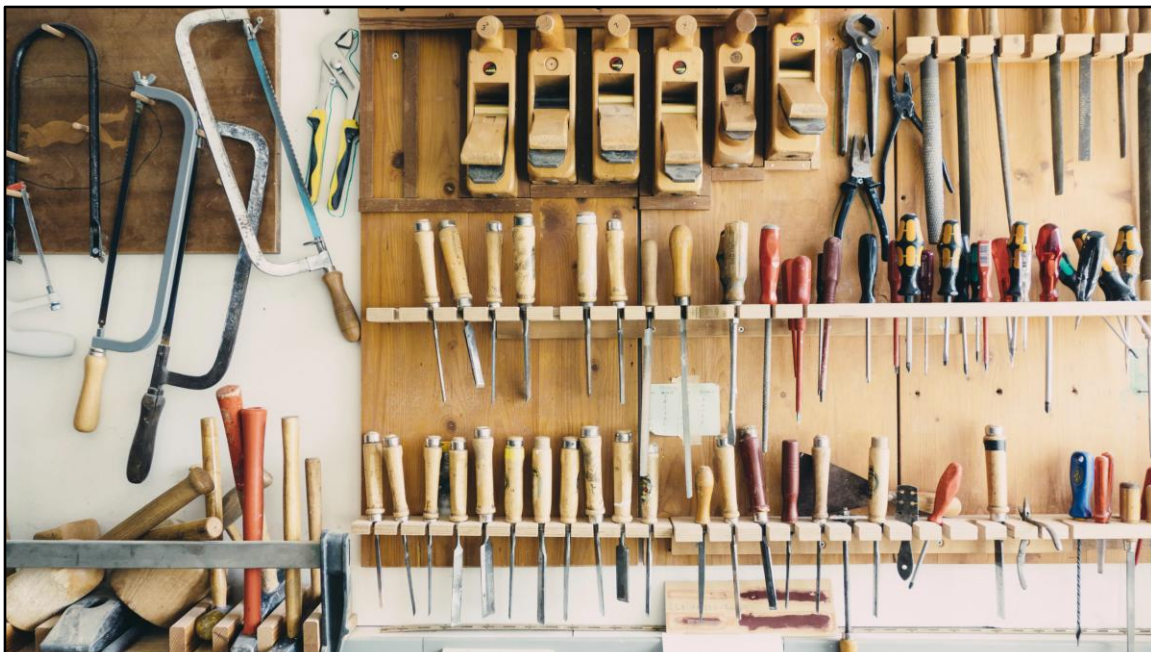


So now, everything in the compiler is an open box. You can take a look at the source code to see how parsing works, what symbols are, etc.

<https://github.com/dotnet/roslyn>



But it's not just about looking at the source code for the compiler. There are numerous ways to build upon and extend the compiler's pipeline.



These APIs let you build powerful tools that you can use in your own code to do things like enforce guidelines and gather metrics on code.

<https://unsplash.com/photos/t5YUoHW6zRo>

Structures

Building Packages and Tools Using the Compiler API

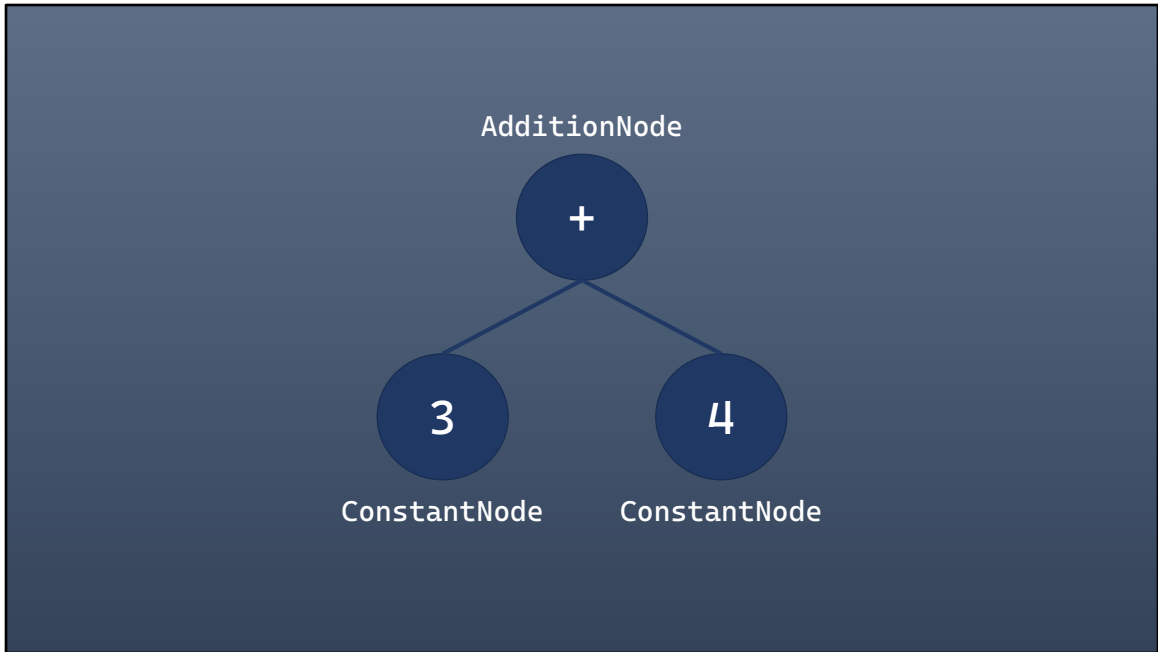


If you've ever read up on how compilers work, you've no doubt run into the tree structure. If you want to build packages that use the Compiler API, it's essential to not only understand the structure of a syntax tree, but related concepts like semantic models, symbols, and operations.

<https://unsplash.com/photos/green-leafed-tree-surrounded-by-fog-during-daytime-S297j2CsdIM>

$$3 + 4$$

Let's start with a very simple example: adding two numbers together

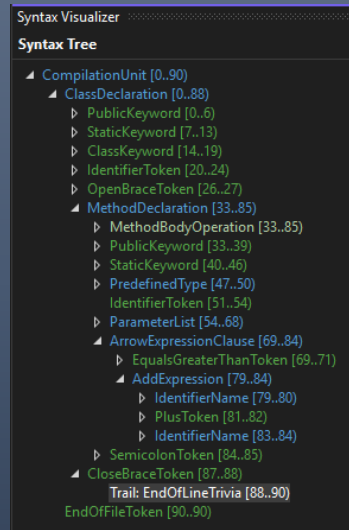


One way to represent this code as data in a tree would be like this. You have an addition node, with two constants: 3 and 4

```
public static class Math
{
    public static int Add(int x, int y) ⇒
        x + y;
}
```

Of course, in C#, nothing is that simple. A generalization of that math problem may look something like this.


```
public static class Math
{
    public static int Add(int x, int y) =>
        x + y;
}
```

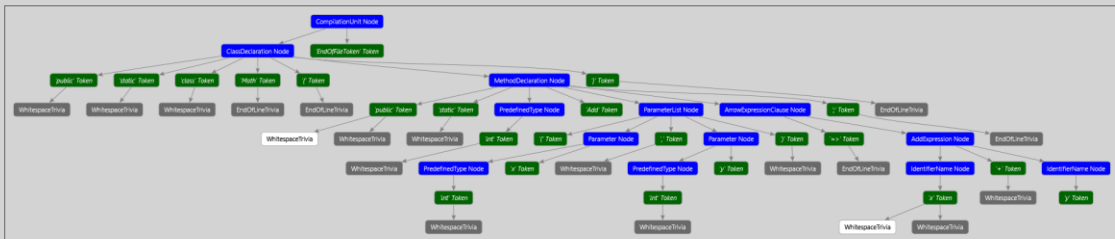


Even a trivial C# implementation can produce a very complicated Roslyn syntax tree. Syntax trees are comprised of syntax nodes, syntax tokens, and syntax trivia. They have full fidelity – meaning, they capture everything in the text.

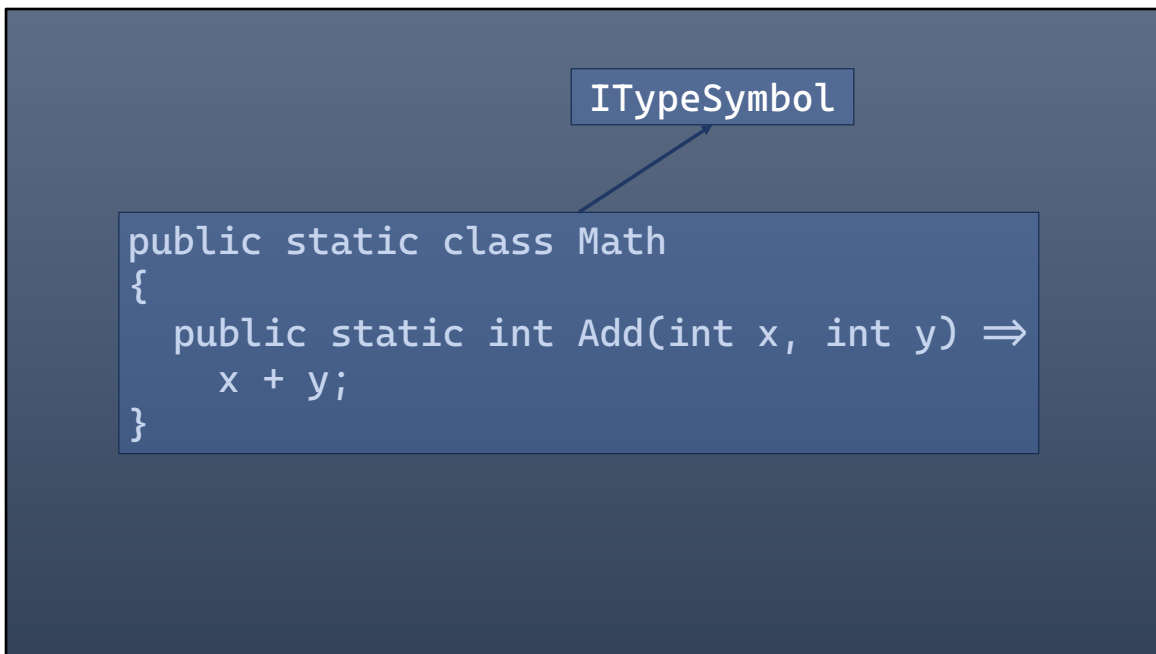
```

public static class Math
{
    public static int Add(int x, int y) =>
        x + y;
}

```

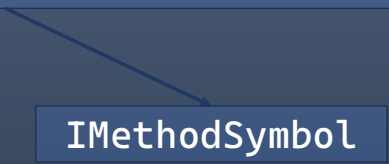


Here's a visual representation of the syntax tree, using a tool called the Syntax Visualizer in Visual Studio.



Syntax trees are the building blocks to understanding what your C# code is actually doing. Typically, you want to know more about the tree than just its' structure. You want semantic meaning. Each tree has a corresponding semantic model where you can get symbols for parts of the tree.

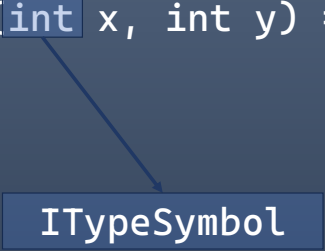
```
public static class Math
{
    public static int Add(int x, int y) =>
        x + y;
}
```



IMethodSymbol

Syntax trees are the building blocks to understanding what your C# code is actually doing. Typically, you want to know more about the tree than just its' structure. You want semantic meaning. Each tree has a corresponding semantic model where you can get symbols for parts of the tree.

```
public static class Math
{
    public static int Add(int x, int y) =>
        x + y;
}
```



ITypeSymbol

Syntax trees are the building blocks to understanding what your C# code is actually doing. Typically, you want to know more about the tree than just its' structure. You want semantic meaning. Each tree has a corresponding semantic model where you can get symbols for parts of the tree.

```
var person = new Person(Guid.NewGuid(), "Jason");
```



ObjectCreation

There are also some generalized operations defined in Roslyn that you can use in certain situations that may make it easier for you to write your tools. In this case, the highlighted code is an ObjectCreation operation

Demo: Using the Compiler API

Building Packages and Tools Using the Compiler API

Let's look at a whole bunch of examples to see how this works:

Transpire: Analyzer + CodeFix, "new Guid()"

AutoDeconstruct: Source Generator + Refactoring

UsingUsings: Tool

Summary

Building Packages and Tools Using the Compiler API



My “call to action” to you is this: climb the mountain. Building artifacts based on the Compiler API can feel daunting.

<https://unsplash.com/photos/5x4U6InVXpc>



If you stick with it, you'll have a great view of your code.

<https://unsplash.com/photos/z0nVqfrOqWA>



I hope I've given you a general roadmap/lay of the land that you can explore in the future. Good luck!

https://unsplash.com/photos/wanderlust-and-explore-concept-old-compass-lying-on-map-top-view-space-for-text-vintage-toned-image-8_B6VHqOkIE

Building Packages and Tools Using the Compiler API

Jason Bock
Staff Software Engineer
Rocket Mortgage

Remember...

- <https://github.com/JasonBock/Transpire>
- <https://github.com/JasonBock/AutoDeconstruct>
- <https://github.com/JasonBock/UsingUsings>
- <https://github.com/JasonBock/Presentations>
- References in the notes on this slide

References

- * [GitHub repository](<https://github.com/dotnet/roslyn>)
- * [A new wave of analyzers in .NET 8](<https://steven-giesel.com/blogPost/1a03a998-e428-4a42-9c30-1cfbf6ac5f4c>)
- * [New analyzers that help you consume APIs the right way](<https://devblogs.microsoft.com/dotnet/announcing-dotnet-7-rc-2/#new-analyzers-that-help-you-consume-apis-the-right-way>)
- * [Overview of .NET source code analysis](<https://docs.microsoft.com/en-us/dotnet/fundamentals/code-analysis/overview>)
- * [Sharing coding style and Roslyn analyzers across projects](<https://www.meziantou.net/sharing-coding-style-and-roslyn-analyzers-across-projects.htm>)
- * [Awesome Roslyn](<https://github.com/ironcev/awesome-roslyn/blob/master/readme.md>)
- * Incremental Roslyn Source Generators In .NET 6: Code Sharing Of The Future
 - * [Part 1](<https://www.thinktecture.com/en/net/roslyn-source-generators-introduction/>)
 - * [Part 2](<https://www.thinktecture.com/en/net/roslyn-source-generators-analyzers-code-fixes/>)

* [Part 3](<https://www.thinktecture.com/en/net/roslyn-source-generators-analyzers-code-fixes-testing/>)

* [Incremental Generators](<https://github.com/dotnet/roslyn/blob/main/docs/features/incremental-generators.md>)

* [Incremental Generators Cookbook](<https://github.com/dotnet/roslyn/blob/main/docs/features/incremental-generators.cookbook.md>)

* [Replacing method calls with Interceptors](<https://andrewlock.net/exploring-the-dotnet-8-preview-changing-method-calls-with-interceptors/>)